



Modelling an Instant Run-off Election Using NetLogo

It is election time in India and the whole country is watching the latest election news, discussing the functioning of EVMs and the code these use, and talking about which party will form the new government. The author takes this opportunity to explore a different kind of electoral system, in which votes are given to candidates in order of preference. We explore the code in such a situation, and look at how the winner is determined.

Intuitively, we understand that an election's outcome will be affected by the nature of the process. Many countries use a run-off between the top two candidates, and the leader of the first round may not make it through the second round. There is also the possibility of an instant run-off in which everyone votes for multiple candidates in the order of preference. Given the time and effort involved in the Indian elections, the instant run-off would seem to be an attractive alternative. However, it is fun to see if we can simulate outcomes by modelling an election. Let us model an instant run-off election with only a second preference vote. The application we will use is NetLogo, which is 'a multi-agent programmable modelling environment'.

You can get started with NetLogo at <http://ccl.northwestern.edu/netlogo/>.

NetLogo is based on the concept of patches. We will assume that each patch represents a constituency. Each constituency will have N candidates, the number of which will vary at runtime. Each tick will represent a voter voting.

Each voter will vote twice. The first vote will be for any one candidate, and the second one for any other candidate.

For the sake of simplicity, voting will be random. However, we will configure the preferred candidates, who will have double the chance of getting a vote.

Figure 1 displays the interface for this model. The *Setup* button initialises the model. The *Go* button counts the votes and shows the results based on the first preference votes. The button for processing the second preference votes will display the result (after the second preference votes have been processed).

The data structures

Keeping track of first preference votes is easy. For each constituency, we need an array or a list with a vote count for each candidate. Keeping track of second preference votes is trickier. We need to be able to count them and

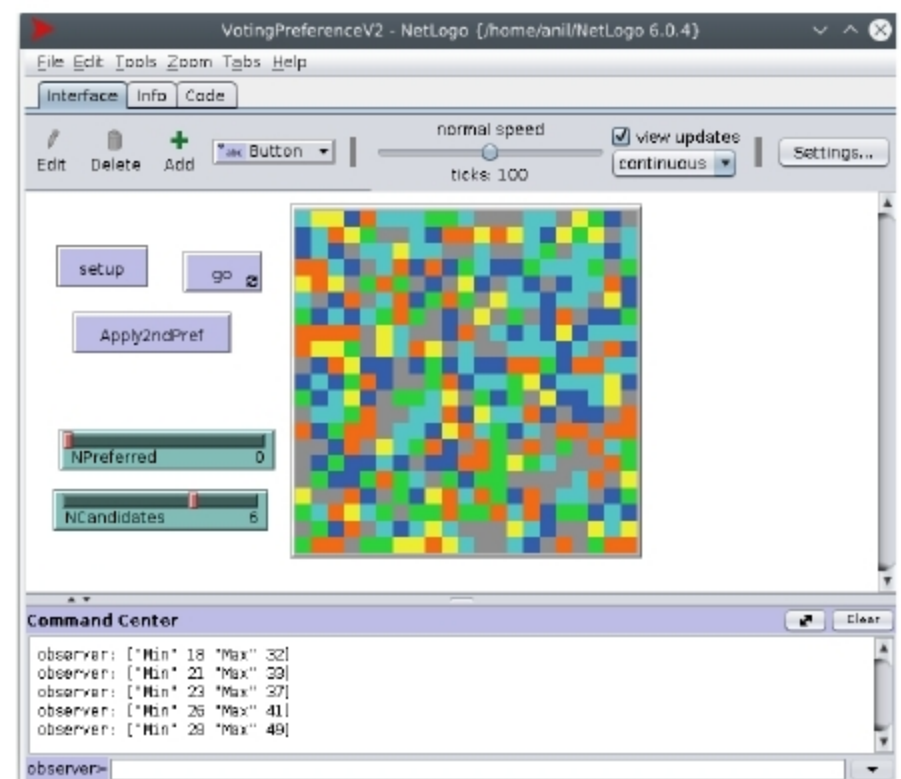


Figure 1: An interface for the simulation. The result shown is for six candidates, whereby each has an equal chance. The colour of a square indicates the winner of the patch.

check if the first preference candidate has been eliminated. Hence, this will need to be a list of lists, in which each list represents the votes cast for a candidate, given that the first preference vote was for a specific candidate.

The setup code

The code is written on the 'Code' tab. If you remember Logo from your school days, you can get a feel for the programming style. The coding style is heavily oriented towards functional list processing paradigms. If you have programmed in Python, you will need to keep looking at the syntax but programming will not be that difficult.

```
globals [NVoters]
```

```
patches-own [votes nextPref candidate winner loser listc]
```

The *patches-own* are variables that are for each patch,